Instructor: Dr. Natalia Chaudhry

Software Quality Engineering, SEF21

Dated: 23-10-2024

SQA Lab#1 : Static code analysis

Create a Sample Java Project:

Open Eclipse and create a new Java project named "SpotBugsDemo".
Create a new Java class named "Main.java" within the project (right clicking src folder).
Start writing code and dont forget to mention package name in a first line

Introduce a Simple Bug

Inside the Main.java class, introduce a simple bug such as a null pointer dereference:

```
public class Main {
  public static void main(String[] args) {
    String str = null;
    System.out.println(str.length()); // Bug: NullPointerException
  }
}
```

Run SpotBugs Analysis:

Right-click on the project "SpotBugsDemo" in the Project Explorer.
Select "SpotBugs" from the context menu, then choose "Find Bugs".

- If running Findbugs does not show up violations then configure spotBugs

To show bug explorer:

Go to Window->Show View->Others->FindBugs->Bugs Explorer

Review SpotBugs Report:

SpotBugs will generate a report highlighting the detected bug(s).
Open the SpotBugs report and review the details of the identified issue(s).

Interpret the SpotBugs Report:

Explore the information provided in the SpotBugs report, such as bug category, severity, and location in the code.
Analyze the potential impact of the detected bug and why it's important to fix it.

Fix the Detected Issue:

Modify the code to fix the null pointer dereference bug:

```
public class Main {
  public static void main(String[] args) {
    String str = null;
    if (str != null) { // Fix: Add null check
      System.out.println(str.length());
    }
  }
}
```

Rerun SpotBugs Analysis:

Repeat the SpotBugs analysis on the project to verify that the issue has been resolved.
Ensure that SpotBugs no longer detects the previously identified bug.

Practice with the following bugs one by one. And repeat the above process for each bug that you add and explore.

BUG #1

```
public class SampleClass {
  public void divideByZero() {
    int numerator = 10;
    int denominator = 0;
    int result = numerator / denominator;
    System.out.println("Result: " + result);
  }

  public static void main(String[] args) {
    SampleClass sample = new SampleClass();
    sample.divideByZero();
  }
}
```

BUG #2: Null Pointer Dereference

```
public class NullPointerExample {
  public static void main(String[] args) {
    String str = null;
    int length = str.length(); // This will trigger a null pointer exception
  }
}
```

BUG #3: Unused Fields or Variables:

```
public class UnusedVariableExample {
  private int unusedField;
  public void unusedMethod() {
    int unusedVar = 42;
  }
}
```

BUG #4: Resource Leak:

```
import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
public class ResourceLeakExample {
  public static void main(String[] args) throws IOException {
    InputStream inputStream = new FileInputStream("file.txt");
    // Missing close() method, can lead to resource leak
  }
}
```

BUG #5: Security leak, SQL injection

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
```

```
public class SQLInjectionExample {
  public void updateUserPassword(Connection connection, String username, String newPassword) {
    String query = "UPDATE users SET password='" + newPassword + "' WHERE username='" +
username + "'";

    try {
      PreparedStatement preparedStatement = connection.prepareStatement(query);
      preparedStatement.executeUpdate();
    } catch (SQLException e) {
      // Handle the exception
    }
  }
}
```

BUG #6: Multithreaded program synchronization bug

```
public class MultithreadedCounter {
  private static int counter = 0;
  public static void main(String[] args) {
    Thread thread1 = new Thread(() -> {
      for (int i = 0; i < 1000; i++) {
        counter++;
      }
    });
    Thread thread2 = new Thread(() -> {
      for (int i = 0; i < 1000; i++) {
        counter++;
      }
    });
```

Handling false positives

```
import edu.umd.cs.findbugs.annotations.SuppressFBWarnings;

public class FindBugsSuppressWarningsExample {

  // Suppress FindBugs warning for null pointer dereference
  @SuppressFBWarnings(value = "NP_NULL_ON_SOME_PATH_FROM_RETURN_VALUE",
          justification = "Just an example, not a real issue")
  public void methodWithFindBugsIssue(String str) {
    if (str.equals("test")) { // Potential null pointer dereference here
      System.out.println("String is test");
    }
  }

  public static void main(String[] args) {
    FindBugsSuppressWarningsExample example = new FindBugsSuppressWarningsExample();
    example.methodWithFindBugsIssue(null); // Will trigger a FindBugs warning
  }
}
```